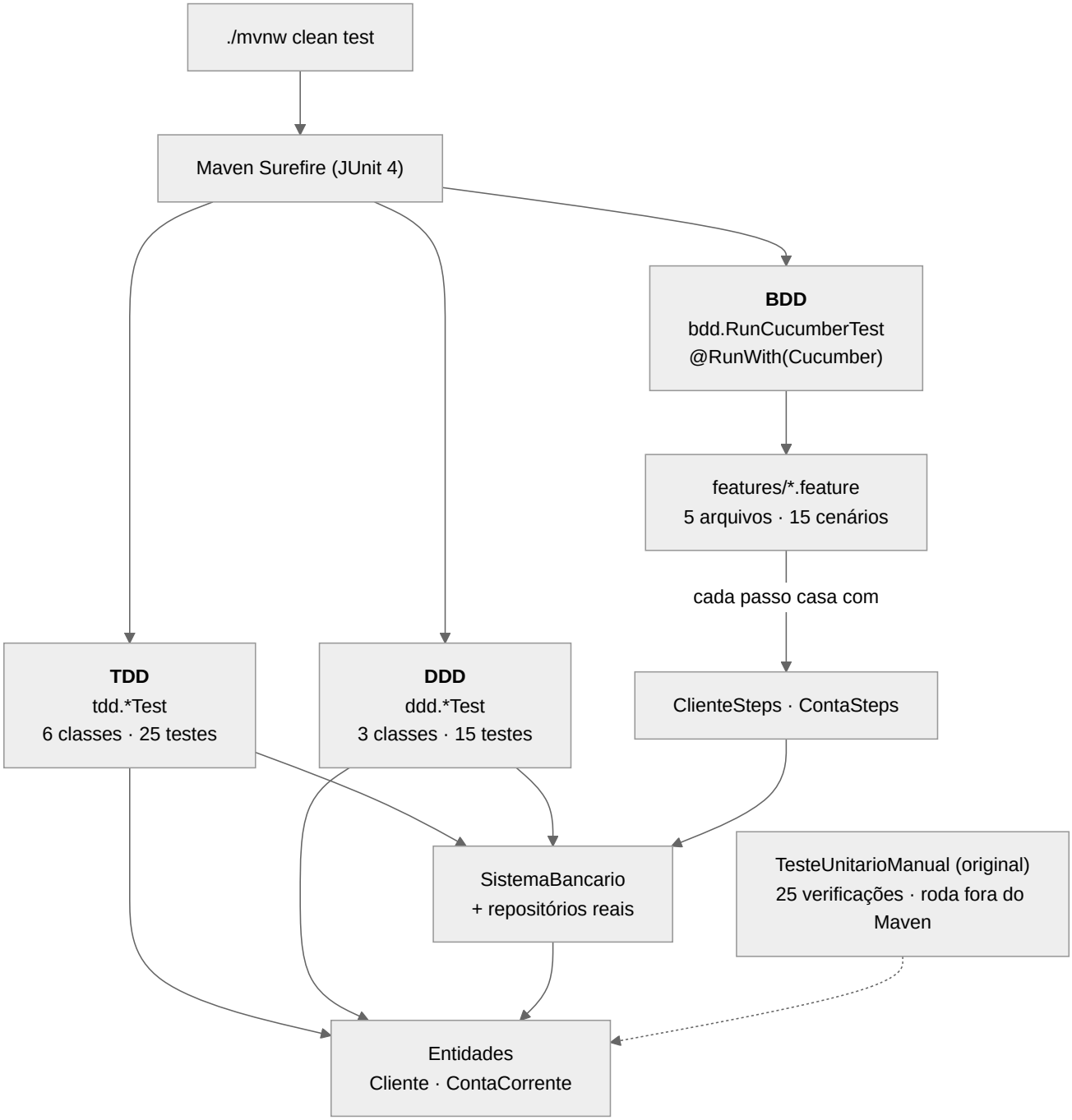
10 — Arquitetura dos testes (TDD, BDD e DDD)

Este documento explica **como os testes estão organizados e executados**: onde fica cada tipo, que parte do sistema cada um exercita, como o Maven encontra e roda tudo, e como cada requisito se liga ao seu teste. O conteúdo de cada ciclo, cenário e decisão de modelagem está nos docs 01, 02 e 03; aqui o foco é a estrutura.

1. Visão geral



Tipo	Pasta	Tecnologia	Quem escreve a especificação	Nº	Pergunta que responde
TDD	`test/java/tdd/`	JUnit 4	Desenvolvedor, em Java	25 testes	"Como implementar esta regra, um passo por vez?"

Tipo	Pasta	Tecnologia	Quem escreve a especificação	Nº	Pergunta que responde
BDD	`test/resources/features/` + `test/java/bdd/`	Cucumber 7 + JUnit 4	Linguagem de negócio (Gherkin em português)	15 cenários	"O que o sistema deve fazer, do ponto de vista do usuário?"
DDD	`test/java/ddd/`	JUnit 4	Desenvolvedor, com nomes do domínio	15 testes	"A regra de domínio está representada e continua valendo?"
Manual (original)	`src/teste/`	`main` + `System.out`	Atividade anterior	25 verificações	Regressão do comportamento original

Resultado final: **55 testes JUnit** (25 + 15 + 15 cenários), todos passando, mais 25/25 no teste manual (`evidencias/54-suite-final-apos-menu-completo.txt`).

2. Como o Maven encontra e executa os testes

O `pom.xml` redefine as pastas porque o projeto não usa o layout padrão:

```
<sourceDirectory>src</sourceDirectory>
<testSourceDirectory>test/java</testSourceDirectory>
<testResources><testResource><directory>test/resources</directory></testResource></testResources>
```

O **Surefire** (plugin de testes do Maven) executa toda classe cujo nome termina em `Test`. Por isso:

- todas as classes de `tdd/` e `ddd/` são executadas diretamente como testes JUnit;
- em `bdd/`, só `RunCucumberTest` é executada pelo Surefire. `ClienteSteps` e `ContaSteps` **não** terminam em `Test`; elas são carregadas pelo Cucumber;
- os `.feature` são copiados para `target/test-classes/features`, por isso o runner os acha com `classpath:features`;
- o `TesteUnitarioManual` está em `src/` (código de produção) e **não** é executado pelo Surefire; roda à parte com `java -cp target/classes teste.TesteUnitarioManual`.

Executar só uma parte:

```
./mvnw test -Dtest='tdd.*'            # só TDD
./mvnw test -Dtest='ddd.*'            # só DDD
./mvnw test -Dtest=RunCucumberTest    # só BDD
./mvnw test -Dtest=TransferenciaTest  # uma classe
```

3. Testes TDD

3.1 Estrutura

```
test/java/tdd/
├── CadastroClienteTest.java           req. 2
├── ValidacaoIdadeClienteTest.java     req. 3
├── CadastroContaCorrenteTest.java     req. 9
├── TransferenciaTest.java             req. 12
├── UnicidadeIdentificadorTest.java    req. 2 e 9 (após auditoria)
└── SaldoFinitoTest.java               req. 9 e 12 (após 2ª auditoria)
```

Cada classe de teste corresponde a **uma regra** (ou um requisito), não a uma classe de produção. Isso é intencional: no TDD o teste nasce da regra, e o código de produção é consequência.

3.2 Anatomia de um teste TDD

```
@Before
public void criarContas() {                       // fixture: objetos novos a cada teste
    origem  = new ContaCorrente(1, 100.0, true);
    destino = new ContaCorrente(2, 0.0, true);
}

@Test
public void naoDeveTransferirValorNegativo() {    // nome = regra em linguagem natural
    // Act
    boolean transferiu = origem.transferir(destino, -50.0);

    // Assert
    assertFalse(transferiu);
    assertSaldosInalterados();                    // helper extraído num REFACTOR
}
```

Convenções usadas em todos os arquivos:

- **Nome** no formato `deve…` / `naoDeve…`, que lê como a regra.
- Blocos **Arrange / Act / Assert** comentados.
- `@Before` cria o estado do zero; nenhum teste depende de outro nem da ordem de execução.
- Exceções verificadas com `assertThrows` (JUnit 4.13), conferindo também que o estado **não mudou** depois da exceção (ex.: `naoDevePermitirAlterarIdadeParaForaDaFaixa` confere que a idade continua 30).
- Comparação de `double` sempre com `DELTA = 0.001`.

3.3 Em que nível cada teste TDD atua

Classe	Alvo direto	Por quê
`ValidacaoIdadeClienteTest`	`Cliente`	A regra é invariante da entidade; não precisa de repositório
`TransferenciaTest`	`ContaCorrente.transferir`	A regra da transferência mora na conta (ver doc 09)
`SaldoFinitoTest`	`ContaCorrente`	Invariante da entidade
`CadastroClienteTest` / `CadastroContaCorrenteTest`	`SistemaBancario`	O caso de uso "cadastrar" é da fachada
`UnicidadeIdentificadorTest`	`GerenciadoraContas`	A unicidade tem que valer em **qualquer** caminho de escrita, não só pela fachada

Nenhum teste usa *mock*: os objetos são pequenos, em memória e rápidos, então o teste com as classes reais é mais simples e mais confiável.

3.4 Como o ciclo RED → GREEN → REFACTOR foi registrado

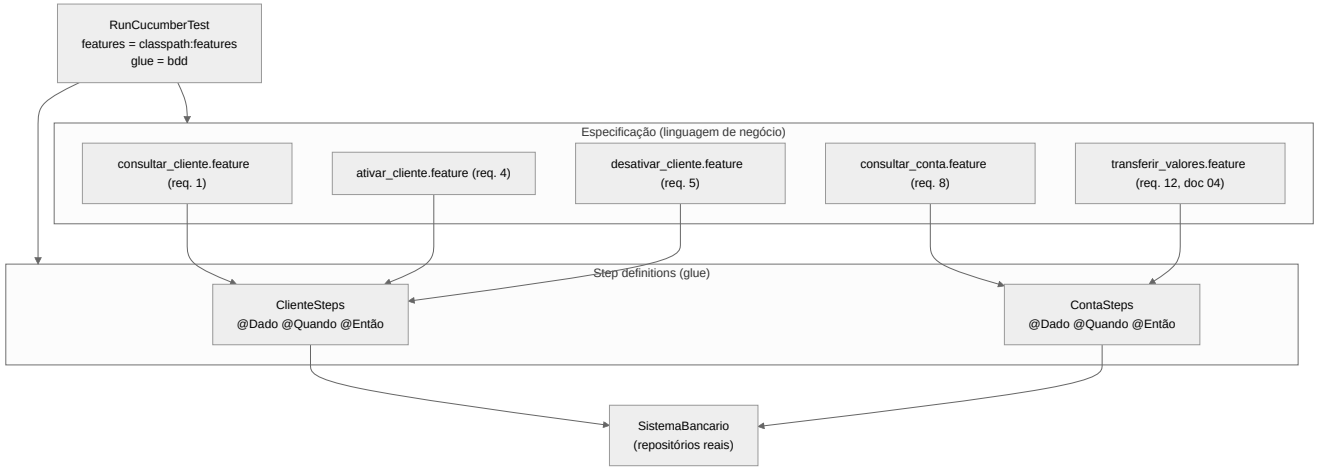
O diretório não é repositório Git, então cada execução foi salva em `evidencias/`, numerada em ordem cronológica:

```
01-req02-red-compilacao.txt           RED  (não compila: método não existe)
02-req02-red-deveCadastrarClienteValido.txt   RED  (compila, assert falha)
03-req02-green-deveCadastrarClienteValido.txt GREEN
...
06-req02-refactor-suite-completa.txt  REFACTOR (suíte inteira + teste manual)
```

Padrão: **RED e GREEN rodam só a classe do requisito** (`-Dtest=...`); **REFACTOR roda a suíte inteira**, para provar que a refatoração não quebrou nada em outro lugar.

4. Testes BDD

4.1 Estrutura em três peças



1. `.feature` — o comportamento em Gherkin, em português (`# language: pt`): `Funcionalidade`, `Cenário`, `Contexto`, `Esquema do Cenário`, `Exemplos`, `Dado`, `Quando`, `Então`, `E`.
2. **Step definitions** — métodos Java anotados com a frase do passo. O Cucumber casa o texto do `.feature` com a expressão da anotação e extrai os parâmetros (`{int}`, `{string}`, `{double}`).
3. **Runner** — `RunCucumberTest`, uma classe vazia com `@RunWith(Cucumber.class)`. É a ponte com o JUnit: cada cenário aparece como um teste no relatório do Surefire.

4.2 Do texto ao código

```
Quando for solicitada a ativação do cliente de id 1
```

casa com

```
@Quando("for solicitada a ativação do cliente de id {int}")
public void forSolicitadaAAtivacao(int id) {
    resultadoOperacao = sistema.ativarCliente(id);
}
```

O mapeamento dos papéis:

Palavra	Papel	O que o step faz
Dado	preparar o mundo	cadastra cliente/conta, muda a situação
Quando	executar **uma** ação	chama uma operação do `SistemaBancario` e guarda o resultado num campo
Então	verificar	`assert*` sobre o resultado guardado ou sobre o estado consultado

4.3 Decisões de arquitetura do BDD

- **Os steps só falam com o `SistemaBancario`** (a fachada), nunca direto com repositório. O cenário descreve o comportamento visto de fora; se a implementação interna mudar, o cenário não muda. Foi isso que permitiu refatorar o domínio no DDD sem tocar nos `.feature`.
- **Isolamento por cenário**: o Cucumber cria uma instância nova de cada classe de steps para cada cenário. Como o `SistemaBancario` é criado no inicializador do campo, todo cenário começa com um banco vazio.
- **Estado compartilhado entre passos** do mesmo cenário fica em campos da classe de steps (`clienteConsultado`, `resultadoOperacao`, `transferenciaRealizada`).
- **Passos reutilizáveis**: "a operação deverá ser concluída com sucesso" serve para ativar e desativar; "que existe uma conta corrente cadastrada..." serve para consulta e transferência.
- **Números em formato brasileiro**: com `# language: pt`, `{double}` lê `100,00` como `100.0` (conferido; ver doc 02).

- **Nível de teste**: os cenários passam por fachada + repositório + entidade reais, sem *mocks*. São o equivalente mais próximo de **teste de integração** nesta entrega.

4.4 Cenários por requisito

Requisito	Arquivo	Cenários
1	`consultar_cliente.feature`	existente · inexistente
4	`ativar_cliente.feature`	inativo → ativo · já ativo (idempotente) · inexistente
5	`desativar_cliente.feature`	ativo → inativo · já inativo (idempotente) · inexistente
8	`consultar_conta.feature`	conta ativa · conta inativa · inexistente
12 (integração)	`transferir_valores.feature`	sucesso · saldo insuficiente · valor inválido (`0,00` e `-50,00` via Esquema)

Total: 2 + 3 + 3 + 3 + 4 = **15 cenários, 70 passos**.

5. Testes DDD

5.1 Estrutura

```
test/java/ddd/
├── SituacaoClienteTest.java          req. 6
├── SituacaoContaCorrenteTest.java    req. 10
└── RemocaoTest.java                  req. 7 e 11
```

5.2 O que muda em relação ao TDD

Tecnicamente são testes JUnit iguais aos de TDD. A diferença está no **objetivo** e no **vocabulário**:

- **Objetivo**: não é conduzir a implementação passo a passo, e sim provar que a **regra de domínio** está representada e continua valendo depois da remodelagem. Por isso foram escritos antes da modelagem (não compilavam, `evidencias/28`) e passaram depois (`evidencias/29`).
- **Vocabulário**: os asserts usam os conceitos do domínio, não tipos técnicos. Compare:

```
// antes (técnico)
assertFalse(cliente.isAtivo());
// DDD (linguagem ubíqua)
assertEquals(SituacaoCliente.INATIVO, cliente.getSituacao());
```

- **Dois níveis em cada classe**: testes direto na **entidade** (`cliente.desativar()` → `INATIVO`) e testes na **fachada** (`sistema.verificarSituacaoCliente(99)` → `Optional.empty()`). O primeiro nível valida o comportamento do conceito; o segundo valida a regra "inexistente ≠ inativo", que só existe no caso de uso.
- **Testes que documentam decisões**: `removerClienteNaoRemoveAContaCorrenteAssociada` não testa uma regra nova; ele fixa uma decisão de modelagem (a relação cliente → conta é só um número) para que ninguém a mude sem perceber.

6. Comparação lado a lado

	TDD	BDD	DDD
Artefato principal	classe de teste JUnit	arquivo `.feature`	classes de domínio + testes JUnit
Linguagem da especificação	Java	Gherkin (português)	Java com nomes do negócio
Ponto de entrada no sistema	entidade ou fachada	só fachada	entidade e fachada
Granularidade	uma regra por teste	um comportamento por cenário	um conceito/invariante por grupo de testes

	TDD	BDD	DDD
Quando foi escrito	antes do código, um por vez	antes (req. 12) ou como especificação executável de comportamento existente (1, 4, 5, 8)	antes da remodelagem
Leitor principal	desenvolvedor	usuário, professor, analista	desenvolvedor e especialista do domínio
Execução	Surefire direto	Surefire → `RunCucumberTest` → Cucumber	Surefire direto

7. Matriz de rastreabilidade — os 12 requisitos

`Requisito → Comportamento / regra → Cenário ou teste → Código de produção`

#	Requisito	Abordagem	Regra / comportamento	Teste(s)	Código exercitado
1	Consultar cliente	BDD	cliente existente é retornado; inexistente → "não encontrado"	`consultar_cliente.feature` (2 cenários)	`SistemaBancario.consultarCliente`
2	Cadastrar cliente	TDD	cadastro válido é consultável; id duplicado é rejeitado	`CadastroClienteTest` (2), `UnicidadeIdentificadorTest` (4)	`SistemaBancario.cadastrarCliente` → `RepositorioBase.adiciona`
3	Validar idade	TDD	idade em [18, 65] na criação e na alteração	`ValidacaoIdadeClienteTest` (6)	`Cliente.validarIdade`, construtor, `setIdade`
4	Ativar cliente	BDD	inativo → ativo; já ativo continua ativo; inexistente rejeitado	`ativar_cliente.feature` (3)	`SistemaBancario.ativarCliente` → `Cliente.ativar`
5	Desativar cliente	BDD	ativo → inativo; já inativo continua; inexistente rejeitado	`desativar_cliente.feature` (3)	`SistemaBancario.desativarCliente` → `Cliente.desativar`
6	Situação do cliente	DDD	ATIVO/INATIVO como conceito; inexistente ≠ inativo	`SituacaoClienteTest` (5)	`SituacaoCliente`, `Cliente.getSituacao`, `SistemaBancario.verificarSituacaoCliente`
7	Remover cliente	DDD	remove existente; rejeita inexistente; não remove a conta	`RemocaoTest` (3)	`SistemaBancario.removerCliente` → `RepositorioBase.remove`
8	Consultar conta	BDD	retorna conta com saldo e situação; inexistente → "não encontrada"	`consultar_conta.feature` (3)	`SistemaBancario.consultarConta`
9	Cadastrar conta	TDD	cadastro válido; número duplicado rejeitado; saldo finito	`CadastroContaCorrenteTest` (2), `UnicidadeIdentificadorTest` (4), `SaldoFinitoTest` (3)	`SistemaBancario.cadastrarConta`, `ContaCorrente.validarSaldo`
10	Situação da conta	DDD	ATIVA/INATIVA como conceito; inexistente ≠ inativa	`SituacaoContaCorrenteTest` (5)	`SituacaoConta`, `ContaCorrente.getSituacao`, `SistemaBancario.verificarSituacaoConta`
11	Remover conta	DDD	remove existente; rejeita inexistente	`RemocaoTest` (2)	`SistemaBancario.removerConta` → `RepositorioBase.remove`

#	Requisito	Abordagem	Regra / comportamento	Teste(s)	Código exercitado
12	Transferir	TDD (+ BDD e DDD no doc 04)	saldo suficiente; rejeita insuficiente, ≤ 0, NaN, destino nulo, estouro; nada muda se rejeitada	`TransferenciaTest` (7), `SaldoFinitoTest`, `transferir_valores.feature` (4)	`SistemaBancario.transferir` → `GerenciadoraContas.transfereValor` → `ContaCorrente.transferir`

Alguns requisitos aparecem em mais de uma abordagem de forma indireta: os cenários BDD de ativar/desativar também confirmam a modelagem DDD de situação (o `Então` verifica `getSituacao()`), e o `TesteUnitarioManual` continua cobrindo remoção, situação e transferência pelo caminho antigo.

8. Relação com os testes da atividade anterior

O `TesteUnitarioManual` (25 verificações) foi **mantido sem alterações** e continua passando. Ele funciona como rede de segurança de regressão: toda mudança de TDD, BDD e DDD foi feita sem quebrar o comportamento que ele verifica. O mapeamento item a item entre o planejamento anterior e os testes atuais está em `06-correcoes-pos-auditoria.pdf`, seção 11.